

Geração de Link para Arquivos EDI (Externo)

Visão Geral

Esta API permite **gerar links temporários** para download de arquivos EDI (Vendas, Pagamentos, Saldo, NRC e PIX) para um **merchant** específico, dentro de um intervalo de datas e tipos de processamento (D - Diário, R - Reprocessamento, M - Mensal).

Fonte: Coleção Postman anexada.



extc-serv-edi-link...n_collection.json

Base URL e Endpoint

SANDBOX: POST <https://apihtml-internet.cielo.com.br/cielo-extc-serv-edi-link-exp-sandbox/extc-serv-edi-link-external/v1/link/generate>

PRODUÇÃO: POST <https://api-internet.cielo.com.br/cielo-extc-serv-edi-link-exp-external/extc-serv-edi-link-external/v1/link/generate>

Autenticação

- **OAuth 2.0 — Client Credentials**

Obtenha o `access_token` via endpoint de OAuth e use `Authorization: Bearer <access_token>` nas chamadas protegidas.

Endpoint de Token:

SANDBOX: POST <https://apihtml-internet.cielo.com.br/cielo-security-sys-web-hml/oauth/v2/MulesoftHML/protocol/openid-connect/token>

PRODUÇÃO: POST https://api-internet.cielo.com.br/cielo-security-sys-web/oauth/v2/MulesoftPRD/protocol/openid-connect/token

Content-Type: application/x-www-form-urlencoded

Parâmetros: client_id, client_secret, grant_type=client_credentials

Resposta (exemplo):

JSON

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUIi...\"",
  "expires_in": 600,
  "token_type": "Bearer",
  "scope": "email profile"
}
```

Importante: Além do token, o serviço requer o header X-Signature (HMAC). Assinatura inválida resulta em **401 Unauthorized (Invalid HMAC)**.

Headers da Requisição

```
Content-Type: application/json
Authorization: Bearer <access_token>
X-Signature: <assinatura_hmac>
```

- Content-Type define que o corpo é JSON.
- Authorization é o Bearer Token obtido via OAuth.
- X-Signature é a assinatura digital/HMAC da requisição.

Body (JSON) — Campos e Regras

Modelo de Requisição

JSON

```
{
  "merchantCode": "1234567890",
  "fileType": [3, 4, 9, 15, 16],
  "processType": ["D", "R"],
  "startDate": "2025-10-06",
  "endDate": "2025-10-06"
}
```

Descrição dos Campos

- merchantCode (*string, obrigatório*): Código da **matriz EDI**.
- fileType (*array[int], obrigatório*): Tipos de arquivo EDI. Valores:
 - 3: **Vendas**

- 4: **Pagamentos**
 - 9: **Saldo**
 - 15: **NRC**
 - 16: **PIX**
- `processType` (*array[string], obrigatório*): Tipos de processamento. Valores:
 - "D": **Diário**
 - "R": **Reprocessamento**
 - "M": **Mensal**
- `startDate` (*string yyyy-MM-dd, obrigatório*): Data inicial.
- `endDate` (*string yyyy-MM-dd, obrigatório*): Data final.

Observações:

- Datas inválidas ou fora da regra podem resultar em **422 Unprocessable Entity**.
- Ausência de campos obrigatórios resulta em **400 Bad Request** (ex.: `merchantCode` ausente).

Assinatura HMAC — x-Signature (Exemplo em Java)

Conforme exemplo abaixo **assina o JSON completo** da requisição, em **UTF-8**, usando **HMAC-SHA256** e codificando o resultado em **Base64**.

Utilitário: Geração do HMAC-SHA256 (Base64)

Java

```
package br.com.comerc.meuservico.utils;

import br.com.comerc.meuservico.exceptions.UnprocessableEntityException;
import java.nio.charset.StandardCharsets;
import java.security.InvalidKeyException;
import java.security.NoSuchAlgorithmException;
import java.util.Base64;
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import lombok.experimental.UtilityClass;

@UtilityClass
public final class HmacGenerator {

    /**
     * Gera a assinatura HMAC-SHA256 em Base64 para o 'data' informado.
     *
     * @param hmacKey Chave secreta HMAC (UTF-8)
     * @param data String canônica a ser assinada (UTF-8)
     * @return Assinatura em Base64 para uso no header X-Signature
     */
    public static String generate(String hmacKey, String data) {
        try {
            Mac mac = Mac.getInstance("HmacSHA256");
```

```

        mac.init(new
SecretKeySpec(hmacKey.getBytes(StandardCharsets.UTF_8), "HmacSHA256"));
        byte[] hmacBytes =
mac.doFinal(data.getBytes(StandardCharsets.UTF_8));
        return Base64.getEncoder().encodeToString(hmacBytes);
    } catch (NoSuchAlgorithmException | InvalidKeyException e) {
        throw new UnprocessableEntityException("Failed to generate
HMAC");
    }
}
}

```

Exemplo de Uso ao Chamar o Endpoint

Java

```

// 1) Monte o JSON exatamente como será enviado no body:
String requestBody = ""
{
    "merchantCode": "1234567890",
    "fileType": [3,4,9,15],
    "processType": ["D","R"],
    "startDate": "2025-10-22",
    "endDate": "2025-10-22"
}
"";

// 2) Gere a assinatura HMAC (assumindo que o corpo JSON é o string-to-sign):
String hmacKey = System.getenv("HMAC_SECRET"); // mantenha em Secrets
Manager/Vault
String xSignature = HmacGenerator.generate(hmacKey, requestBody);

// 3) Envie a requisição com os headers obrigatórios:
HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api-internet.cielo.com.br/cielo-extc-serv-edi-
link-exp-external/extc-serv-edi-link-external/v1/link/generate"))
    .header("Content-Type", "application/json")
    .header("Authorization", "Bearer " + accessToken) // obtido no OAuth
    .header("X-Signature", xSignature)
    .POST(HttpRequest.BodyPublishers.ofString(requestBody))
    .build();

HttpResponse<String> response = httpClient.send(request,
HttpResponse.BodyHandlers.ofString());
System.out.println(response.statusCode());
System.out.println(response.body());

```

Dicas para evitar 401 Invalid HMAC:

- Use **exatamente** o mesmo requestBody para gerar a assinatura e para enviar no HTTP.
- Normalize quebras de linha, espaços e ordem de campos (se houver canônico).
- Confirme o **algoritmo** e **codificação** (HmacSHA256 + Base64).
- Armazene a chave HMAC em **AWS Secrets Manager** (conforme seu stack em EKS/Terraform).

Fonte: exemplos/erros presentes na coleção.

Exemplos de Uso (cURL)

```
curl -X POST \
  'https://api-internet.cielo.com.br/cielo-extc-serv-edi-link-exp-external/extc-serv-edi-link-external/v1/link/generate' \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer '"$ACCESS_TOKEN"' \
  -H 'X-Signature: '"$HMAC_SIGNATURE"' \
  -d '{
    "merchantCode": "1234567890",
    "fileType": [3,4,9,15],
    "processType": ["D","R"],
    "startDate": "2025-10-22",
    "endDate": "2025-10-22"
  }'
```

Exemplo de Resposta (200 OK):

```
{
  "links": [
    {
      "url": "https://link.com",
      "duration": 15,
      "merchantCode": "1234567890",
      "fileType": 3,
      "generationDate": "2025-10-22"
    }
  ]
}
```

Códigos de Resposta e Erros Comuns

- **200 OK** — Links gerados com sucesso.
- **400 Bad Request** — Body malformado ou campo obrigatório ausente (ex.: merchantCode). Também pode ocorrer quando o token não é enviado (JWT Token is required.).
- **401 Unauthorized** — **HMAC inválido** ou token inválido/ausente. Mensagem típica: Invalid HMAC.
- **422 Unprocessable Entity** — Erro de validação (datas fora da regra, etc.).

Contratos de Saída

Objeto `Link` (por item)

JSON

```
{
  "url": "https://link.com",
  "duration": 15,
  "merchantCode": "1234567890",
  "fileType": 3,
  "generationDate": "2025-10-22"
}
```

- `url`: Link de download (temporário).
- `duration`: Validade do link (minutos).
- `merchantCode`: Merchant.
- `fileType`: Tipo de arquivo EDI.
- `generationDate`: Data de geração.

Segurança

- **Bearer Token (OAuth 2.0)**: Obrigatório em `Authorization`.
- **HMAC (X-Signature)**: Obrigatório para integridade/autenticidade; cálculo e chave conforme governança interna.

Boas Práticas (Operação/Infra)

- **Validade dos Links**: Consumir antes de `duration` expirar.
- **Observabilidade**: Logar campos principais e status HTTP; útil para troubleshooting.
- **Segurança em Trânsito**: Sempre **HTTPS**.

Troubleshooting

- **401 — Invalid HMAC**: Revisar conteúdo assinado, chave, algoritmo e header.**400 — JWT Token is required / Campo obrigatório ausente**: Garantir token em `Authorization` e todos os campos obrigatórios.
- **422 — Erro de Validação**: Checar intervalo de datas e tipos válidos.

Checklist de Integração

1. Obter `access_token` via OAuth (client credentials).
2. Calcular e enviar `X-Signature` (HMAC) no header.
3. Montar body JSON com campos obrigatórios.
4. Enviar `POST` e capturar `links` na resposta.
5. Baixar arquivos antes do término de `duration`.